VisionMate

Your eyes, your voice, your independence

M2: Mid-submission | Due 4:00 AM, 11th October

1. Current Approach and Changes Since M1

Since M1 we turned the plan into a working, deployable prototype of about 2,800 lines of HTML, CSS, JavaScript and TypeScript. The approach is now **safety-first and provider-backed**: the browser handles the camera, microphone, speech output, location and the interface; every secret and third-party AI call sits behind a server-side function; and every AI answer is spoken with a reminder to verify it. The current architecture has five parts:

- **Frontend:** a no-build web app in HTML5, CSS3 and vanilla JavaScript modules, with large high-contrast controls, a spoken onboarding flow, a live region for screen readers, a web manifest, and a Leaflet map. It supports seven languages: English, Hindi, Tamil, Telugu, Kannada, Malayalam and Punjabi.
- **Identity:** Firebase Authentication with Google and email sign-in. The app reports an error rather than faking a login when Firebase is not configured.
- **Backend:** three Supabase Edge Functions (Deno/TypeScript): *gemini-assist* for scene description and questions, *anuvaadhub-inference* for speech recognition, speech synthesis, OCR and translation, and *bhashini-translate* as a second translation provider. Keys live only in server secrets.
- **Data:** Supabase Postgres with eight tables (profiles, settings, conversations, messages, notes, contacts, reminders, activity events), all protected by row-level security so a user can reach only their own rows.
- **Assistant logic:** a rule-based command router handles notes, timers, reminders, voice and theme changes, saved-contact calls, opening allow-listed sites, location and SOS; anything else goes to Gemini.

Changes since M1. We compare the submitted M1 with the code as it stands today:

- **Backend changed.** M1 proposed a Node.js gateway plus Python services for OCR, detection and depth. We replaced both with serverless Supabase functions, which keeps keys off the browser, removes servers to host, and fits the timeline.
- **Language services changed.** M1 used Bhashini for speech, translation, OCR and voice. We now use the Anuvaadhub model catalogue for speech recognition, speech synthesis, OCR and translation, keep Bhashini as an optional translation route, and use the browser's own speech engines for live listening and speaking.
- **Vision scope reduced on purpose.** M1 planned on-device detection and depth (YOLO, MiDaS). The prototype instead captures a single camera frame for Gemini description or OCR. Continuous detection is not built.
- **Safety stance tightened.** M1 promised directional walking guidance, Sonar and Risk Radar. After review we made the prototype refuse walking, pothole and road-safety questions, and we instructed the AI never to call a path or crossing safe, because unvalidated detection could endanger users.
- **Added beyond M1:** account sign-in, cloud-synced notes, contacts and reminders, saved conversations, an activity log, battery warnings, a confirm-first phone dialer, a location map with an SOS location link, voice selection, light and dark themes, and three more languages.
- **Kept from M1:** voice-first design, plain web stack, server-held keys, privacy by default, SOS and battery alerts, the "verify this" confidence wording, and the path from localhost to a deployed URL to an Android APK.
- **Not yet built:** Mate Sonar, Risk Radar, Hazard Map, Scene-Aware Auto Mode, Spatial Memory, object detection and depth, Mate Circle, the offline ladder, the wake word, a service worker and the Android wrapper.

2. Experiments Run So Far and Their Results

All results below were measured on the current code base. Live provider trials need valid keys and are listed under the remaining plan.

- **Provider catalogue filtering.** The access file holds 1,543 approved model entries (1,435 translation, 44 OCR, 35 speech synthesis, 29 speech recognition). Filtering to our seven languages and four tasks produced **63 usable routes**: 42 translation and 7 each for recognition, synthesis and OCR. Every one of our languages is covered for every task.

- **Access-window finding.** All 63 routes expire on 13 October 2026, 04:30 UTC. The function refuses calls after that time, so the final demo must happen before then or the catalogue must be refreshed.
- **Command routing.** We replayed 17 spoken-style phrases through the router's actual rules: notes, timers, reminders, theme change, SOS, "close WhatsApp", opening apps, location, reading a menu, walking questions and a general question. **All 17 routed as intended**, and the pothole, road-safety and "guide me" questions were correctly refused.
- **Code health.** All 15 JavaScript modules and scripts pass a syntax check, and the offline fallback replies return the expected text for all five test inputs.
- **Provider limits discovered.** Translation accepts 1 to 50 words, speech synthesis up to 30 words with letters, numbers and spaces only, and recorded speech up to 20 seconds or 5 MB. Gemini calls fall back to the next configured model on a 404, 429 or 503 error. We built input checks and clear spoken errors around each limit.
- **Weakness found.** The offline fallback replies carry fixed confidence numbers (91 to 99 percent) that are not measured. They must be replaced by real scoring before we present confidence as a feature.
- **Security review.** The repository ignores the environment file and the access file, provider secrets are set only on the server, the browser holds just the public project keys that are designed to be public, and every database table enforces row-level ownership. One risk remains: the Supabase session is anonymous and per device, so records do not yet follow a person across devices until Firebase and Supabase identities are linked.

3. Progress on Testing

M1 defined five test families. Current status against that plan:

- **Done:** command-routing checks, syntax checks, provider-catalogue validation, and safety-behaviour checks (walking and hazard prompts refused, AI instructed not to declare any path safe).
- **Written, not yet run live:** the browser-to-function-to-provider chain, the database policies and the three functions are in place, with live key trials pending.
- **Findings so far:** every walking and hazard test phrase was refused as designed, errors are spoken in plain language, and the main gaps are measurement (no accuracy or latency numbers yet) and untested live providers.
- **Not started:** accuracy measurement (speech word error rate, OCR character error rate, translation quality in English, Hindi, Tamil and Telugu), stage-by-stage latency against the 700 ms and 3 s targets, low-light and noise tests, and consented trials with blind and low-vision users.

4. Plan for the Remaining Time

- **Immediately:** push all current code to `main` with `.env` and the access file excluded, and confirm no secret is in the history.
- **Accuracy and latency runs:** test 20 or more samples per language for OCR, recognition and translation, record the numbers, and report them in the final submission.
- **Trust Meter:** replace fixed confidence values with real scoring, and add a second pass for medicine and currency reads.
- **Offline and install:** add a service worker for an offline shell, then wrap the site as an Android APK with Capacitor.
- **Experimental vision:** try in-browser object detection with clock-direction output and a Web Audio Sonar, labelled experimental and kept away from walking guidance until validated.
- **User validation:** run short trials with volunteers, record task success and a usability score, and fix the top problems found.

Risks we are tracking. The following could affect the final demo, and each has a planned mitigation:

- **Provider access expiry:** the Anuvaadhub window closes on 13 October, so we will record demo evidence early and keep browser speech as a fallback.
- **AI errors:** Gemini and OCR can misread or invent details, so every answer keeps its spoken "please verify" warning and high-stakes uses stay out of scope.
- **Browser limits:** a website cannot close other apps, run reliable background alarms or detect hazards unaided, and speech voices vary by device; these are stated in the app and the Android wrapper is our route past them.